CGT 390 Lab 9 API and Data Design Report — Backend Draft

Mahina Yasutake | Gatherly | October 22, 2026

Live prototype: cgt-390-project-seven.vercel.app

Backend branch: github.com/MahinaYasutake/CGT390_Project/tree/codex/lab9-backend-database

Status: The API and database passed local tests. The live URL still serves the browser-local version until a persistent cloud database is connected and deployment is verified.

1. Interaction and contract

The task is to save Clay Club to a shortlist, see it after a reload, and remove it later. The revised prototype sends this action to a server API backed by an SQLite-compatible database.

Entry point

The Save button in app.js sends POST /api/saved with JSON {"eventId":"clay-club"}. api/saved.js validates the request, derives an anonymous visitor from an HttpOnly cookie, writes the database row, and returns the current IDs. GET /api/saved reads the list; DELETE /api/saved?eventId=clay-club removes one item.

Annotated flow

Save button → POST /api/saved → request validation and visitor cookie → events table ID check → saved_events insert → JSON response → Saved Events view. The public catalog supplies display details; saved_events stores the visitor-event relationship.

Contract

POST accepts exactly one field, eventId, as a string. A new save returns HTTP 201 with eventId, saved:true, and ids; a duplicate returns 200 with the same single saved row. DELETE returns 200 with saved:false and updated ids. A malformed body returns 400 INVALID_BODY; an unknown event returns 404 EVENT_NOT_FOUND; database unavailability returns 503 SAVE_SERVICE_UNAVAILABLE.

2. Data and access decisions

The events table contains catalog IDs. saved_events contains visitor_id, event_id, and created_at. Its (visitor_id, event_id) primary key prevents duplicates; event_id has a foreign key to events. The shared catalog.js array supplies public event details and seeds valid IDs in the database. The database is the source of truth for saved relationships.

The server creates a random visitor cookie and stores a hash of it as visitor_id. It never accepts a visitor ID from request JSON. Every read and write is scoped to that derived ID, so one anonymous browser cannot target another visitor's rows through the API. The cookie is HttpOnly and SameSite=Lax. This is anonymous ownership, not a signed-in account; clearing the cookie loses access to the list. No email, token, or payment field is returned.

3. Testing evidence

The local API ran against a temporary real database on October 22. Each prediction was checked against the HTTP response and a subsequent read. The sanitized test log contains every case and omits visitor cookie values. The browser UI was also checked by saving on Explore, reloading, and viewing Saved Events.

T1 Successful action

Prediction: POST clay-club as visitor A returns 201; GET returns one saved ID. **Actual:** 201 {"eventId":"clay-club","saved":true,"ids":["clay-club"]}; GET returned the same ID. The local browser showed Saved 1 after reload.

T2 Invalid input

Prediction: POST eventId 42 is rejected without a write. **Actual:** 400 {"error":"INVALID_BODY"}; the saved list stayed ["clay-club"]. POST not-real returned 404 EVENT_NOT_FOUND and also left the list unchanged.

T3 Access boundary

Prediction: visitor B starts empty and cannot remove visitor A's save. **Actual:** B's GET returned {"ids":[]}; B's DELETE clay-club left A's GET at {"ids":["clay-club"]}. A POST body with an extra visitorId field returned 400. SQL WHERE visitor_id = ? explains the isolation.

T4 Task-specific edge case

Prediction: a repeated POST clay-club must not create a duplicate. **Actual:** HTTP 200 returned the same one-item ids list. The database primary key enforced uniqueness. A direct insert with a nonexistent event ID was rejected by the foreign key.

4. Findings and response

The earlier live prototype used only localStorage. After the professor clarified that a backend and database are expected, the revised prototype added a server endpoint, persistent-row model, server-side validation, visitor-scoped queries, database constraints, and a one-time transfer of earlier saved IDs. The new local API test passed all asserted cases, and four local browser checks passed, including two-tab saving and migration.

Remaining work: connect a persistent cloud database to Vercel, deploy this branch, and repeat the HTTP and browser tests at the live URL. Until then, the live prototype still uses localStorage. The anonymous cookie is not an account, and reservations, concurrent availability checks, and third-party analytics delivery remain outside this tested scope.

5. Peer review and AI use

Peer review

Pending. No classmate has yet proposed and run an additional case. Share the new API contract and ask a classmate to select and try a different test. Record their input, expected result, actual status and response, and any fix or clarification.

AI use

ChatGPT/Codex reviewed the code, built and ran the tests, implemented repairs, and drafted this report. The claims above were checked against the code, browser results, and stored values. I reviewed the document, decided on which action to investigate, and drafted parts of this report.